

PlaceMind: Role-Based Architecture & Engineering Guidelines

This document outlines the architectural responsibilities, technical stacks, and specific implementation requirements for each engineering role involved in the PlaceMind platform. The architecture emphasizes high performance, secure microservices, and an exceptional user experience.

1. Frontend Engineering (React / TypeScript)

The Frontend Engineer is responsible for building a scalable, performant Single Page Application (SPA) that communicates with the Node.js API Gateway.

Domain	Technology Stack	Key Responsibilities
Core Framework	React 18+, TypeScript, Vite	Ensure strict type safety across all components and API responses.
State & Data Fetching	Redux Toolkit / Zustand, React Query	Manage global auth state and cache API data for the mock interview and resume scoring dashboards.

Key Implementation Details

- Implement protected routing using React Router v6 to ensure unauthenticated users cannot access dashboard or AI features.
- Establish an Axios interceptor to automatically attach JWT tokens to outgoing requests and handle token refresh logic seamlessly.

2. UI/UX Design (Figma & Design Systems)

The UI/UX Designer bridges the gap between raw data and user interaction, ensuring a seamless and visually cohesive experience across the platform.

Domain	Technology Stack	Key Responsibilities
Design System	Figma, Material Design 3 Kit	Build and maintain a centralized component library mirroring Tailwind CSS configurations.
Prototyping	Interactive Figma Flows	Map out complex user journeys, such as the multi-step mock interview process.

Key Implementation Details

- Design accessible data visualizations for AI outputs (e.g., radar charts for skill matching, progress bars for career roadmaps).
- Establish standard spacing, typography, and color tokens that can be directly mapped to the frontend team's Tailwind configurations.

3. Backend Engineering (Node.js Gateway)

The Backend Engineer constructs the central API gateway, managing database transactions, authentication, and routing traffic to the AI microservices.

Domain	Technology Stack	Key Responsibilities
API Gateway	Node.js, Express.js	Handle all client-facing REST APIs and proxy heavy AI requests to the FastAPI service.

Domain	Technology Stack	Key Responsibilities
Data Layer	PostgreSQL, Prisma ORM, Redis	Design robust relational schemas for user profiles, interview histories, and roadmaps.

Key Implementation Details

- Implement robust rate limiting utilizing Redis to prevent abuse of the backend endpoints.
- Handle initial file uploads (e.g., resumes, videos) and stream them securely to the MinIO/S3 storage buckets before notifying the AI service.

```
// Example: Proxying request to FastAPI
app.post('/api/analyze-resume', authenticateJWT, async (req, res) => {
  const aiResponse = await axios.post('http://ai-service:8000/resume/score', req.body);
  return res.json(aiResponse.data);
});
```

4. AI / ML Engineering (Python / FastAPI)

The AI/ML Engineer is responsible for processing heavy computational workloads, interfacing with Large Language Models, and generating predictive analytics.

Domain	Technology Stack	Key Responsibilities
Microservice API	Python 3.12, FastAPI, Pydantic	Expose high-performance asynchronous endpoints for the Node Gateway.

Domain	Technology Stack	Key Responsibilities
LLM & ML Integration	Google Gemini API, XGBoost	Design strict prompts for the mock interview engine and train hiring prediction models.

Key Implementation Details

- Ensure all data structures returned by the Gemini LLM are strictly validated using Pydantic models before returning payloads to the Node.js layer.
- Develop graph traversal algorithms in Python for the Career Roadmap Generator to determine prerequisite mappings efficiently.

Note: All microservices are containerized via Docker Compose for uniform local development environments.